

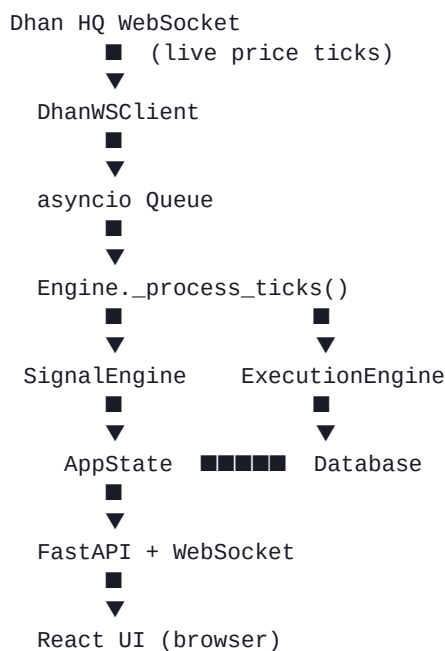
Market Sentinel

NSE Options IV-Accumulation Trading Daemon — Architecture & Code Walkthrough

What It Does (The Strategy)

Market Sentinel watches **Nifty 50 options** in real time. When OTM options start moving disproportionately fast compared to the spot price — a sign of implied-volatility accumulation or smart-money positioning — it opens a paper trade to capture that move. It exits when the signal reverses, when stops trigger, or at market close.

System Architecture



Layer by Layer

1. core/ws_client.py — Price Feed

Connects to the **Dhan HQ live WebSocket** using your API credentials. Subscribes to five instruments: Nifty spot, ATM call, ATM put, OTM call, and OTM put. Every price update is parsed into a **Tick** object and pushed into an `asyncio.Queue` — the bridge between the live feed and the trading engine. Auto-reconnects on disconnect; calls `instrument_provider()` on every reconnect so subscriptions always reflect the latest strikes after a 30-minute recalibration.

2. core/instruments.py — Strike Selection

Every **30 minutes**, calls the Dhan option-chain REST API and picks the best strikes:

Strike	Selection rule
ATM	Nearest strike to current Nifty spot (rounded to 50)
OTM Call	Strike whose Black-Scholes delta is closest to 0.10–0.15
OTM Put	Strike whose $ \text{delta} $ is closest to 0.10–0.15

Uses a Black-Scholes delta formula for each strike. After calibration it pushes the new symbol assignments to the SignalEngine and the WebSocket client resubscribes.

3. core/signal.py — The Math Engine

The core of the strategy. For every incoming tick:

Step 1 — Rolling 30-second windows

A sliding price window per symbol. Computes ΔSpot (spot price change over last 30s) and ΔOTM (OTM option price change over last 30s).

Step 2 — Sideways filter

If $|\Delta\text{Spot}| < 2$ points, market is sideways. Skip — ratio is unreliable when the underlying barely moved.

Step 3 — Ratio

```
ratio =  $|\Delta\text{OTM} / \Delta\text{Spot}|$ 
```

If OTM options are moving 3× faster than spot, ratio = 3.0. A high ratio signals unusual implied-volatility accumulation.

Step 4 — Z-score (20-minute baseline)

```
Z = (current_ratio - mean) / std_dev
```

Keeps the last 20 minutes of ratio samples. The signal fires only when $Z > 2.0$ (two standard deviations above the recent mean). This makes the signal *self-calibrating* — the threshold adapts to whether the market is currently calm or volatile.

4. core/execution.py — Paper Trade State Machine

Opening a trade

Sizes the position at 20% of virtual capital, computes lots at the current ATM premium, records entry price and timestamp.

Four exit conditions

Exit	Trigger
Divergence collapse	Z-score drops back below 2.0 (signal reversed)
Stop loss	ATM premium drops 30% from entry
Trailing stop	After +20% gain, trails 15% below the running peak
Time stop	15:15 forced close before market end

Kill switch

If cumulative daily loss $\geq 40\%$ of day-start capital, halt all trading for the session.

Cooldown

15-minute lock after any trade closes — prevents overtrading.

5. core/engine.py — The Orchestrator

The main asyncio loop that connects everything. For every tick:

1. Log tick to CSV
2. Update AppState prices (spot, ATM, OTM)
3. Feed SignalEngine -> z_sample_count, z_score, ratio
4. If still in 5-minute warmup -> wait (no trades)
5. If ACTIVE:

- a. Check stop / trailing exit on open trade
- b. Check 15:15 time stop
- c. Check daily-drawdown kill switch
- d. Check new signal -> open trade
- e. Check divergence collapse -> close trade
- f. Sync AppState (capital, P&L, open_trade)

Warmup (5 minutes after start or reconnect): the engine still feeds the SignalEngine so the Z-score baseline is already built when trading begins. No trades open during warmup.

6. core/database.py — Persistence

SQLite (aiosqlite) database with three tables:

Table	Stores
trades	Every paper trade — entry, exit, P&L, exit reason
signal_events	Every signal fired (acted on or skipped)
system_log	Engine start / stop / errors

7. api/ — FastAPI Backend

REST endpoints

Endpoint	Purpose
GET /api/status	Engine state, prices, P&L snapshot
GET /api/trades	Trade history (with CSV download)
GET /api/signals	Signal history
POST /api/config	Change parameters live (no restart)
POST /api/token	Update Dhan API token; triggers WS reconnect

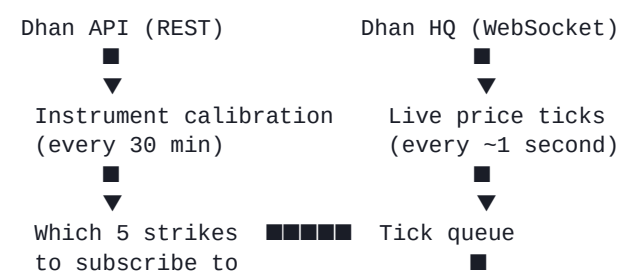
WebSocket /ws/live

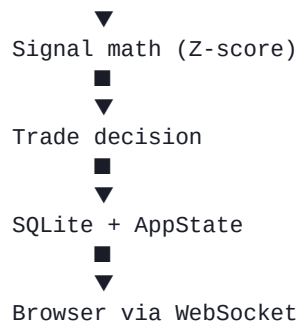
The engine writes to a broadcast queue every tick; FastAPI fans this out to all connected browsers. The UI updates in real time without polling.

8. frontend/ — React Dashboard

Panel	Shows
Header	Engine state, Nifty spot, ATM price, capital, day P&L, uptime
Warmup bar	Fills as Z-score samples accumulate (0→30)
Z-Score Gauge	Current Z-score vs 2.0 threshold
Spot + Z chart	Last 60 s of spot price with Z-score overlay
P&L sparkline	Daily P&L curve
Position Tracker	Open trade details + live unrealised P&L
Signal Feed	Last 30 signals with Z, direction, acted/skipped
Trade Blotter	Full trade history + CSV download
Config Panel	Edit parameters live + update Dhan token

Data Flow Summary





Key Design Decisions

Decision	Why
Single asyncio event loop	Engine + API share the loop & rarr; AppState is in-memory shared, no IPC
Self-calibrating Z-score	Signal threshold adapts to current volatility automatically
Paper trading only	All trades are virtual; no real orders are ever placed
systemd service	Engine runs 24/7 on the server; closing the browser does not stop it
30-min recalibration	ATM/OTM strikes track a trending market without manual intervention
5-min warmup blackout	After reconnect, rebuild the Z-score baseline before resuming trades

Tunable Parameters (config.yaml)

Parameter	Default	Meaning
signal.zscore_threshold	2.0	Z-score level that fires a signal
signal.min_spot_delta	2.0	Sideways filter (points of spot move)
signal.zscore_lookback_minutes	20	Baseline window for the Z-score
signal.min_history_samples	30	Samples needed before Z is trusted
execution.virtual_capital	10,00,000	Starting paper-trade capital (■)
execution.max_position_pct	0.20	% of capital per trade
execution.stop_loss_pct	0.30	Premium stop-loss
execution.trailing_stop_activation_pct	0.20	Profit % to activate trailing
execution.trailing_stop_pct	0.15	Trailing stop distance from peak
execution.cooldown_minutes	15	Lock after a trade closes
execution.daily_drawdown_kill_pct	0.40	Daily loss kill-switch threshold
execution.force_close_time	15:15	Time-stop force close
ws.warmup_seconds	300	Post-reconnect blackout